

Programming 2

CMP1025

Lecture 5 – Arrays

Kenrayan Whittle

Learning Objectives

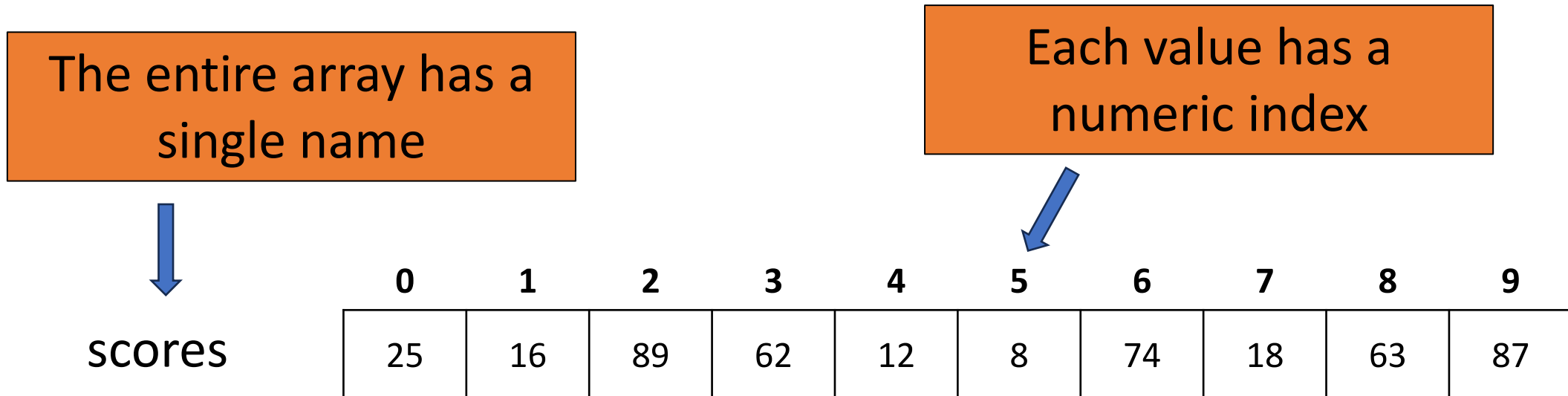
- Use the array data structure to represent lists and tables of values.
- Define an array, initialize an array and refer to individual elements of an array.
- Define symbolic constants.
- Arrays and functions

Introduction to Arrays (1D)

- **Arrays** are data structures consisting of related data items of the same type.
- An array is a group of **homogeneous contiguous** memory locations.
- Arrays are “static” entities in that they remain the same size throughout program execution.
- All elements of the array share the same name and each is individually identified by a number.
- Any one of these elements may be referred to by giving the array’s name followed by the **position number** of the particular element in square brackets ([]).

Arrays (1D)

An array is a variable that can store multiple values of the same type.



An array of size N is indexed from zero to N-1
This array holds 10 values that are indexed from 0 to 9

Array (1D)

- The first element in every array is the **zeroth element**.
- An array name, like other identifiers, can contain only letters, digits and underscores and cannot begin with a digit.
- The position number within square brackets is called an **index** or **subscript**.
- An index must be an integer or an integer expression.
 - For example, if **a = 3** and **b = 2**, then the statement **scores[a + b] += 2;**
 - adds 2 to array element **scores[5]**.

Defining/Declaring Array (1D)

- Arrays occupy space in memory.
- You specify the type of each element and the number of elements the array requires so that the computer can reserve the appropriate amount of memory.
- The following definition reserves **10 elements** for integer array scores, which has indices in the range **0 – 9**.

```
int scores[10];
```

Defining/Declaring Array (1D)

```
int b [100] , x[27];
```

reserves 100 elements for integer array b and 27 elements for integer array x. These arrays have indices in the ranges 0 – 99 and 0 – 26, respectively. Arrays can contain other data types. Like any other variables, uninitialized array elements contain garbage values.

Initializing Array (1D)

The elements of an array can also be initialized when the array is defined by following the definition with an equals sign and braces, { }, containing a comma-separated list of array initializers. If there are **fewer initializers** than elements in the array, the remaining elements are initialized to zero. The elements of the array **numbers** could have been initialized to zero as follows:

// initializes entire array to zeros

```
int numbers[5] = {0};
```


Initializing Array

Arrays are not automatically initialized to zero. You must at least initialize the first element to zero for the remaining elements to be automatically zeroed.

The array definition

```
int numbers[5] = {32, 27, 64, 18, 95, 14};
```

causes a syntax error because there are six initializers and **only** five array elements.

Define & Initializing Array (1D)

If the array size is **omitted** from a definition with an initializer list, the number of elements in the array will be the number of elements in the initializer list.

For example,

```
int numbers[ ] = {1, 2, 3, 4, 5};
```

would create a five-element array initialized with the indicated values.

Define Array (1D)

The **#define preprocessor directive** defines a **symbolic constant** SIZE whose value is 5.

```
#define SIZE 5
```

A symbolic constant is an identifier that's replaced with **replacement text** by the C preprocessor before the program is compiled.

When the program is preprocessed, all occurrences of the symbolic constant SIZE are replaced with the replacement text 5.

Accessing Arrays (1D)

Elements of an array are accessed by their name followed by the position number (index number) contained in square brackets. Below show numbers being assigned to each elements of the array, **numbers**.

numbers[0] = 23;

numbers[1] = 59;

numbers[2] = 102;

numbers[3] = 44;

numbers[4] = 16;

	23	59	102	44	16
Indices →	0	1	2	3	4

printf("4th Element: %d", numbers[3]);

Accessing Arrays (1D)

Using a loop to **traverse** the elements of an array is more efficient than sequentially visiting each element.

```
for (counter = 0; counter < 5; counter++)  
{  
    printf("Enter number: ");  
    scanf("%d", &numbers[counter]);  
}
```

Note the upper bound is 4, not 5. That is, you loop through 0 to 4 to initialize an array with 5 elements

Multidimensional Arrays

- Arrays in C can have multiple indices.
- Typically used to represent **tables** of values consisting of information arranged in **rows** and **columns**.
- To identify an element, two indices must be specified: The first identifies the **row** and the second identifies the **column**.
- Two-dimensional arrays have rows and columns
- Multidimensional arrays can have more than two indices

Multidimensional Arrays

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Diagram illustrating the structure of a 2D array (a) with 3 rows and 4 columns. The array is represented as a grid of elements, each labeled with its row and column indices (e.g., a[row][column]).

Arrows point to the components of the array notation:

- Column index (points to the second index, e.g., 1 in a[2][1])
- Row index (points to the first index, e.g., 2 in a[2][1])
- Array name (points to the variable 'a' in a[2][1])

Multidimensional Arrays

The element in array `a` is identified by an element name of the form

`arr2D[row][column];`

- `arr2D` is the name of the array
- `row` and `column` are the indices that identify each element

Multidimensional array like a one-dimensional (1D) array must be defined before use. To define the array, indicate the name, number of rows and the number of columns.

For example, a two-dimensional array: **`int arr2D[3][4];`**

Defining a 2D Array

arr2D[3][4]

				0
		25		1
40				2
0	1	2	3	

Each element of arr2D is identified by the name combined with the index of the row and the index of the column. To assign 25 to the element coloured red we do the following: **arr2D[1][2] = 25;** While **arr2D[2][0] = 40;** will assign 40 to the cell coloured green.

Calculating the size of the Array

Calculate the size of a multidimensional array by multiplying the size of all the dimensions. For example:

the size of `arr[10][20]` (rows * columns)

$$10 * 20 = 200 \text{ elements}$$

Initializing Multidimensional Arrays

- A multidimensional array can be initialized when it's defined, much like a one-dimensional array.
- For example, a two-dimensional array `int b[2][2]` could be defined and initialized with

```
int b[2][2] = {{1,2}, {3,4}};
```

- The values are grouped by row in braces.
- The values in the first set of braces initialize row 0 and the values in the second set of braces initialize row 1.
- So, the values **1** and **2** initialize elements **b[0][0]** and **b[0][1]**,
- while the values **3** and **4** initialize elements **b[1][0]** and **b[1][1]** respectively.

Initializing Multidimensional Arrays

If there are not enough initializers for a given row, the remaining elements of that row are initialized to 0.

Thus,

```
int b[2][2] = {{1}, {3,4}};
```

would initialize **b[0][0]** to 1, **b[0][1]** to 0, **b[1][0]** to 3 and **b[1][1]** to 4

1	0	0
3	4	1
0	1	

Initializing Multidimensional Arrays

```
int b[2][3] = {1, 2, 3, 4, 5};
```

- If the braces around each sub-list are removed from the array's initializer list, the compiler initializes the elements of the first row followed by the elements of the second row.
- The definition above provides five initializers.
- The initializers are assigned to the first row, then the second row.
- Any elements that do not have an explicit initializer are initialized to zero automatically, so **b[1][2]** is initialized to **0**.

Manipulating 2D Array

To manipulate a 1D array it is common to use a for loop. 2D arrays we use nested for loops.

```
total = 0;
for(row = 0; row < 3; row++){
    for(col = 0; col < 4; col++){
        total += b[row][col];
    }
}
```

1	2	3	4	0
5	6	7	8	1
9	10	11	12	2
0	1	2	3	

Arrays and Functions

When you pass a 1D array to a function, you are passing its address (base address). So, the function works on the original array.

```
void printArray(int arr[ ], int size)
{
    for(counter = 0; counter < size; counter++)
    {
        printf("%d ", arr[i]);
    }
}
```

```
//function call from main()
int numbers[5] = {8,2,7,4,9};
printArray(numbers, 5);
```

Arrays (2D) and Functions

Unlike 1D array, when passing a 2D array, the column size MUST be specified. We pass rows separately or can be handle alternatively

```
#include <stdio.h>

void printMatrix(int matrix[][3], int rows);

int main() {
    int matrix[2][3] = { {1,2,3}, {4,5,6} };

    printMatrix(matrix, 2);

    return 0;
}

void printMatrix(int matrix[][3], int rows)
{
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}
```


Arrays and Functions

```
//unsized array
double calcAverage(double tution[], int size){ //accepts an array and the size of the array
    //function body
}

//sized array
double calcAverage(double tution[7]){ //accepts an array with its size
    //function body
}
```

```
//unsized array
calcAverage(tution, 7); //passes an array and the size of the array

//sized array
calcAverage(tution); //passes an array
```

Arrays (1D) and Functions

```
2
3 //decalaration/proyotype
4 void printArray(double tuition[], int size); //accepts an array of type double, and size of array as int
5
6 int main()
7 { //declaration of index variable
8     int index;
9
10    //call printArray() to print all the elements in the array
11    printArray(tuition, 7);
12
13    //sum all the values in the array that was passed to the function; variable is formal parameter tuition[]
14    for(index = 0; index < size; index++){
15        sum = sum + tuition[index]; //get data from array using array name and position; tuition[index]
16    }
17
18    printf("Sum is %lf\n", sum); //print sum with a meaningful statement
19    return 0;
20 }
21
22 //definition
23 void printArray(double tuition[], int size){ //accepts an array and the size of the array
24     int i; //declaration of index variable
25     //print the array index along with the value at each index in the array
26     for(i=0; i<size; i++){
27         printf("tuition[%lf] = %lf\n", index, tuition[index]); //get data from array
28     }
29 }
30
```

The End...

Next topic: Sorting and Searching Arrays